ClubJoin Portal

1. PROJECT EXPLANATION

1.1 Overview

The Campus ClubJoin Portal is a centralized, interactive web application developed to modernize and streamline student participation in extracurricular activities across a college campus. In collegiate environments, extracurricular clubs—ranging from technology, robotics, and competitive coding to fine arts, cultural events, sports, and social welfare—play an indispensable role in developing students' leadership, collaboration, and professional competencies. However, traditional methods of club discovery and membership sign-up rely on physical notice boards, scattered paper forms, and uncoordinated instant messaging groups, resulting in low student awareness, missed opportunities, and administrative bottlenecks.

ClubJoin establishes a unified, responsive platform where students can seamlessly discover active campus clubs, filter communities by domain and category, review upcoming events, and join clubs with a single click. Simultaneously, the system provides faculty coordinators and student administrators with dedicated management tools to oversee registrations, schedule events, and monitor campus-wide involvement through real-time statistical metrics.

1.2 Objectives

- **Centralized Extracurricular Hub:** Consolidate all campus clubs, activities, and member registrations into a single, dependable collegiate portal.
- **Instant Search & Filtering:** Enable students to locate clubs immediately via live keyword search (name, description) and domain-specific category filters (Technology, Arts, Culture, Sports, Social Service).
- **Student Lifecycle & Self-Service:** Provide secure student account creation, credential verification, and personal dashboards tracking active memberships and join dates.
- **Administrative Oversight & Governance:** Deliver an administrator dashboard with live statistical counters, registered student roster management with deletion controls, and event scheduling.
- **Relational File-Based Data Persistence:** Implement a robust file-based storage architecture using structured JSON files (data.json, data/clubs.json, memberships.json, events.json) with OS-level atomic file locking (LOCK_EX) ensuring concurrency safety.
- **Modern, Accessible, Mobile-First UI:** Deliver a responsive, mobile-optimized interface styled with collegiate blue and indigo palettes without requiring heavy external dependencies.

1.3 Key Functionality & Modules

1. Home Landing Module (index.html): Features a modern hero presentation, collegiate brand identity, responsive navigation with hamburger toggle, core value propositions, and featured club showcases.

2. Club Exploration & Search Module (clubs.html, js/script.js): Provides asynchronous data loading via fetch(), live real-time keyword search, category dropdown filtering, and dynamic card generation with one-click enrollment triggers.

3. Student Registration & Auth Module (register.html, login.html, reg.php, login.php): Implements dual-layer validation with client-side regex checks, server-side sanitization, password hashing via BCRYPT, and session regeneration.

4. Student Dashboard & Membership Portal (dashboard.php, myclubs.php, profile.php): Provides personalized dashboards displaying enrolled clubs with join timestamps, duplicate membership prevention, and student profile inspection.

5. Administrator Management Suite (admin-login.html, admin-dashboard.php, admin-students.php, admin-events.php): Equips administrators with role-based dashboard metrics (Total Students, Total Clubs, Total Memberships, Total Events), full student roster management with deletion capabilities, and event creation tools.

6. Backend Data Persistence Layer (data.json, memberships.json, events.json, data/clubs.json): Maintains relational entity mappings with atomic file locking (LOCK_EX) to prevent race conditions during concurrent user registrations and membership sign-ups.

2. SCREENSHOTS OF THE MAIN MODULE

[Screenshot 1: Home Page Dashboard & Hero Showcase]

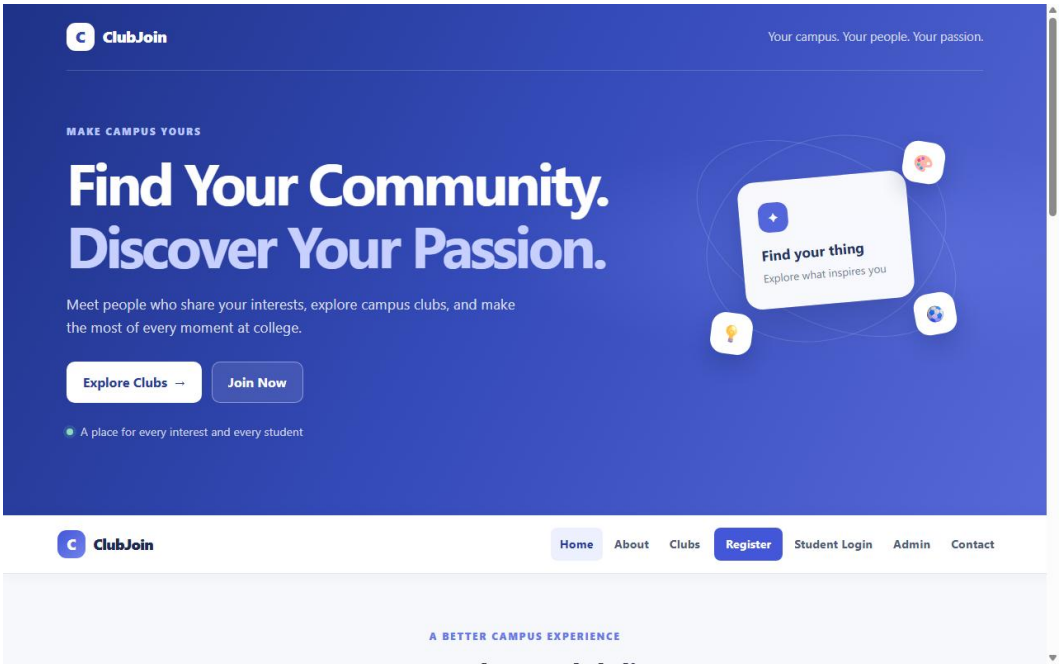


Figure 1.1: ClubJoin Portal Landing Page & Hero Section displaying collegiate branding, navigation tabs, and mission tagline.

[Screenshot 2: Explore Clubs Module with Filter Tabs]

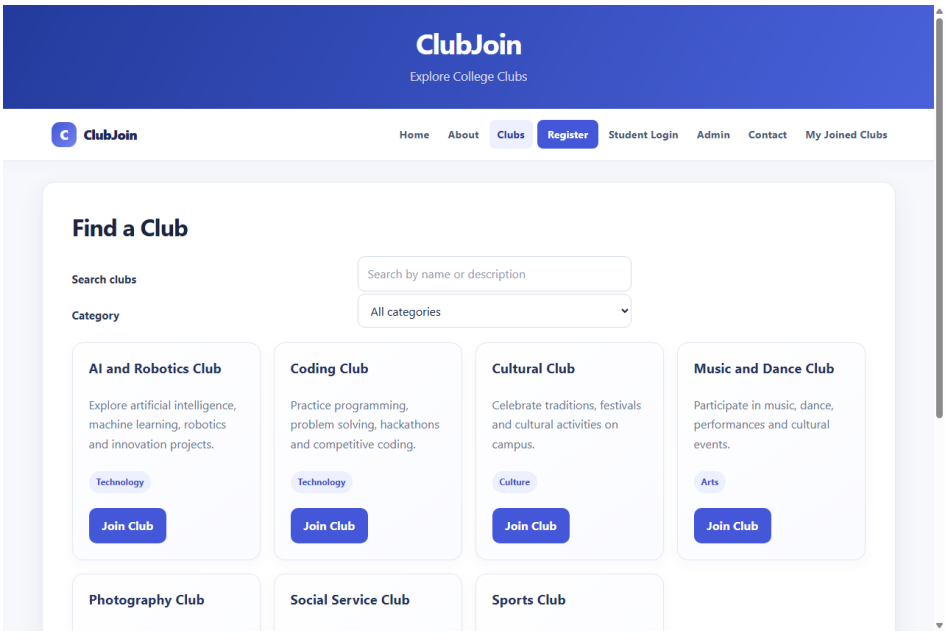


Figure 1.2: Explore Clubs interface featuring real-time keyword search, category dropdown filtering, and dynamic club cards.

[Screenshot 3: Student Account Registration Form]

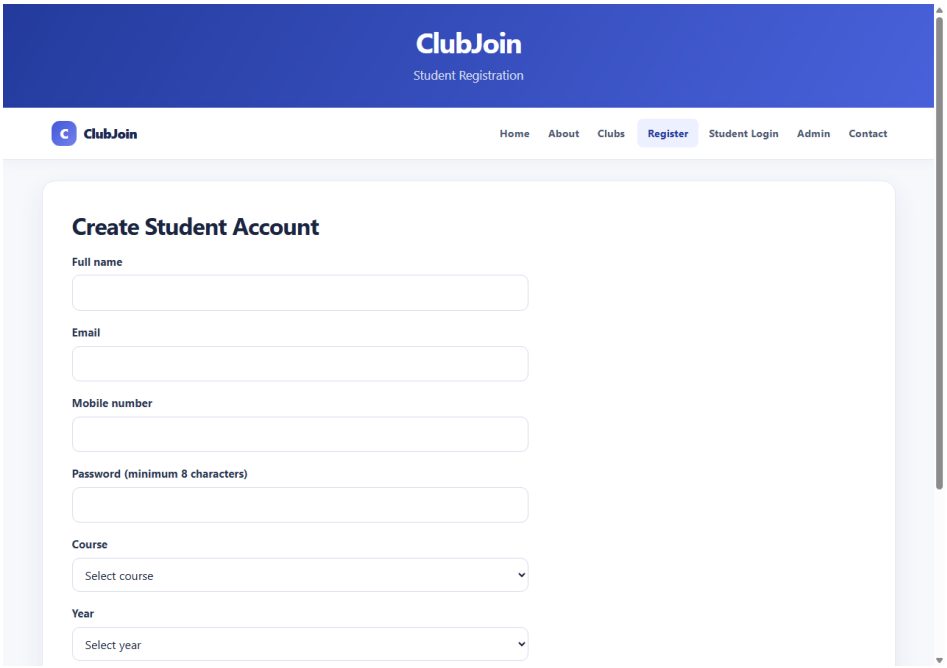


Figure 1.3: Student Account Registration form with client-side regex input validation and academic course/year selection.

[Screenshot 4: Student Dashboard & Enrolled Memberships Module]

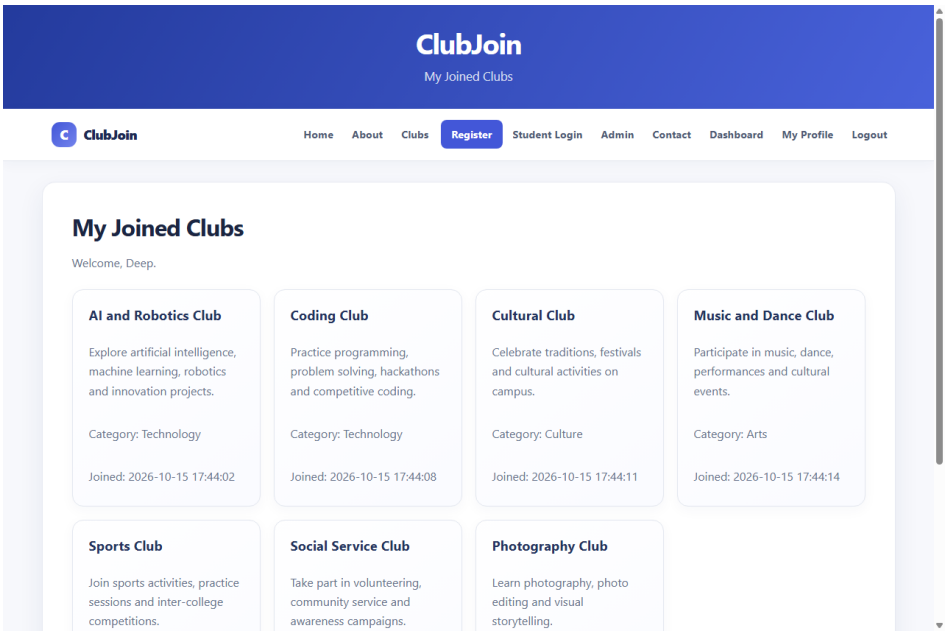


Figure 1.4: Student "My Joined Clubs" Portal displaying enrolled club memberships, categorized tags, and join timestamps.

[Screenshot 5: Administrator Control Dashboard]

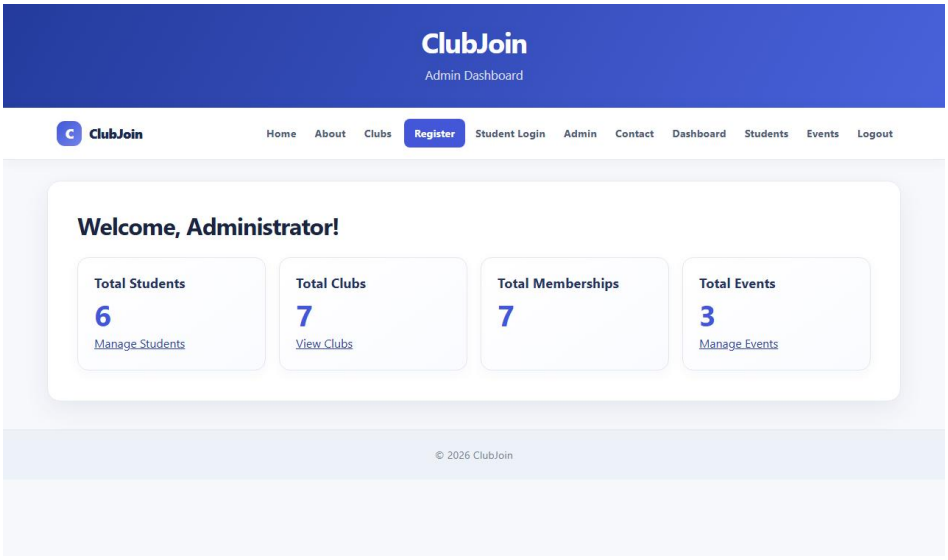


Figure 1.5: Administrator Dashboard displaying real-time statistical counters for Total Students, Total Clubs, Total Memberships, and Total Events.

[Screenshot 6: Administrator Student Management Module]

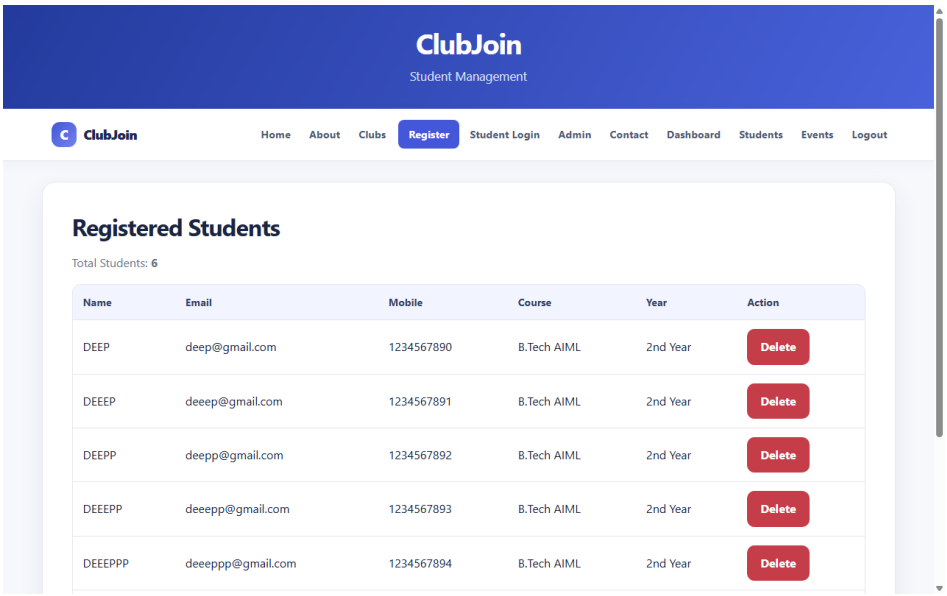


Figure 1.6: Administrator Student Management interface showing registered students data table with deletion actions.

[Screenshot 7: Backend Relational Data Architecture & JSON Records]

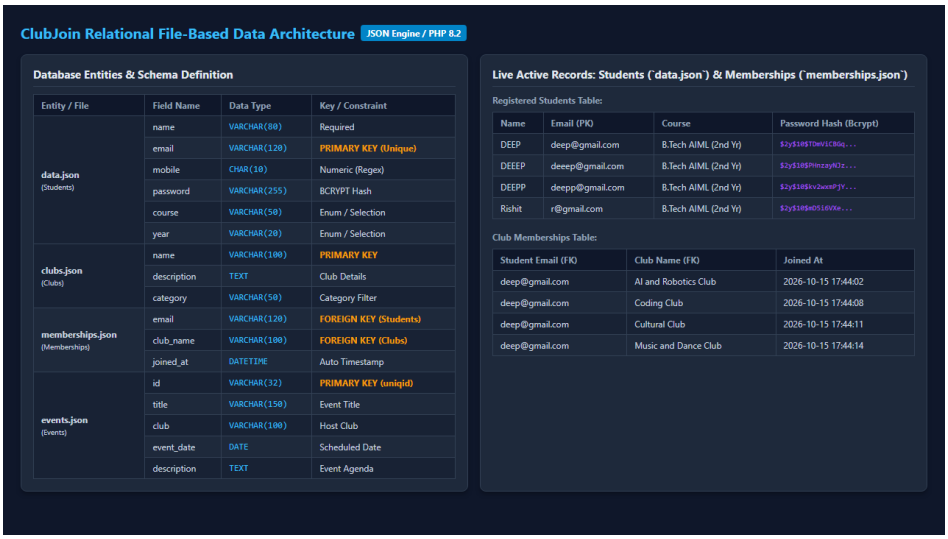


Figure 1.7: ClubJoin Relational Data Architecture showing entity schemas, primary/foreign key mappings, and active backend database records.

3. MAIN MODULE LOGIC CODE

3.1 Front-End Controller Logic (script.js - Dynamic Search & Validation)

javascript

```
// ============================================================================
// CLIENT-SIDE FORM VALIDATION CONTROLLER
// ============================================================================
function validateRegistration() {
    const name = document.getElementById('name').value.trim();
    const email = document.getElementById('email').value.trim();
    const mobile = document.getElementById('mobile').value.trim();
    const password = document.getElementById('password').value;
    const course = document.getElementById('course').value;
    const year = document.getElementById('year').value;
    const terms = document.getElementById('terms').checked;

    if (!/^[a-zA-Z .'-]{2,80}$/.test(name)) {
        alert('Please enter a valid name.'); return false;
    }
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
        alert('Please enter a valid email.'); return false;
    }
    if (!/^[0-9]{10}$/.test(mobile)) {
        alert('Mobile number must contain exactly 10 digits.'); return false;
    }
    if (password.length < 8) {
        alert('Password must contain at least 8 characters.'); return false;
    }
    if (!course || !year) {
        alert('Please select your course and year.'); return false;
    }
    if (!terms) {
        alert('Please accept the terms and conditions.'); return false;
    }
    return true;
}

// ============================================================================
// ASYNCHRONOUS DATA FETCHING & DYNAMIC CLUB FILTERING
// ============================================================================
const clubList = document.getElementById('clubList');
if (clubList) {
    const searchInput = document.getElementById('searchClub');
    const categoryFilter = document.getElementById('categoryFilter');
    const loading = document.getElementById('loadingMessage');
    const noResults = document.getElementById('noResults');
    let allClubs = [];

    // Asynchronously fetch clubs catalog from JSON backend
    fetch('data/clubs.json')
        .then(response => {
            if (!response.ok) throw new Error('Unable to load clubs.');
            return response.json();
        })
        .then(clubs => {
            allClubs = clubs;
            if (loading) loading.hidden = true;
            displayClubs();
        })
        .catch(err => {
            if (loading) loading.hidden = true;
            console.error('Fetch error:', err);
        });
```

```
    // Real-time compound search and category filtering function
    function displayClubs() {
        const term = searchInput ? searchInput.value.toLowerCase().trim() : '';
        const category = categoryFilter ? categoryFilter.value : '';

        const filtered = allClubs.filter(c => {
            const matchesQuery = (c.name || '').toLowerCase().includes(term) ||
                                 (c.description || '').toLowerCase().includes(term);
            const matchesCat = !category || c.category === category;
            return matchesQuery && matchesCat;
        }).sort((a, b) => (a.name || '').localeCompare(b.name || ''));

        clubList.innerHTML = '';
        if (noResults) noResults.hidden = filtered.length !== 0;


 // Dynamic DOM element generation for filtered club cards

        filtered.forEach(club => {
            const article = document.createElement('article');
            article.className = 'club-card';

            const h = document.createElement('h3');
            h.textContent = club.name || 'Club';

            const p = document.createElement('p');
            p.textContent = club.description || '';

            const c = document.createElement('p');
            c.className = 'category-pill';
            c.textContent = club.category || 'Other';

            const form = document.createElement('form');
            form.action = 'join.php';
            form.method = 'post';

            const input = document.createElement('input');
            input.type = 'hidden';
            input.name = 'club_name';
            input.value = club.name || '';

            const button = document.createElement('button');
            button.type = 'submit';
            button.textContent = 'Join Club';

            form.append(input, button);
            article.append(h, p, c, form);
            clubList.appendChild(article);
        });
    }
    if (searchInput) searchInput.addEventListener('input', displayClubs);
    if (categoryFilter) categoryFilter.addEventListener('change', displayClubs);
}

// ============================================================================
// SERVER-SIDE MEMBERSHIP ENROLLMENT CONTROLLER (join.php)
// ============================================================================
<?php
session_start();
if (!isset($_SESSION['student_email'])) { header('Location: login.html'); exit; }
```

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') { header('Location: clubs.html'); exit; }
$clubName = trim($_POST['club_name'] ?? '');
$clubsFile = __DIR__ . '/data/clubs.json';
$membershipFile = __DIR__ . '/memberships.json';

$clubs = json_decode(file_get_contents($clubsFile), true);
$memberships = json_decode(file_get_contents($membershipFile), true);

// Duplicate enrollment verification check
foreach ($memberships as $membership) {
    if (strtolower($membership['email'] ?? '') === strtolower($_SESSION['student_email']) &&
        strtolower($membership['club_name'] ?? '') === strtolower($clubName)) {
        header('Location: clubs.html?status=already_joined'); exit;
    }
}

// Atomic write operation with exclusive file locking (LOCK_EX)
$memberships[] = [
    'email' => $_SESSION['student_email'],
    'club_name' => $clubName,
    'joined_at' => date('Y-m-d H:i:s')
];
file_put_contents($membershipFile, json_encode($memberships, JSON_PRETTY_PRINT), LOCK_EX);
header('Location: clubs.html?status=success'); exit;
?>
```

3.2 Explanation of Main Module Logic

1. Compound Asynchronous Filtering (displayClubs):
The client-side exploration module decouples data delivery from UI presentation. Upon page initialization, the entire club catalogue is asynchronously retrieved via fetch('data/clubs.json') and cached in client memory (allClubs). The filter pipeline evaluates two criteria simultaneously upon every input or change event:

- Text Substring Matching: Evaluates whether user keystrokes match either the club title or description using case-insensitive .includes().
- Domain Category Filtering: Verifies whether the club belongs to the selected category (Technology, Arts, Culture, Sports, Social Service).

Matching records are alphabetically sorted using localeCompare() and immediately converted into DOM elements without triggering full page reloads.

2. Dual-Layer Validation & Sanitization (validateRegistration & reg.php):
Security and data cleanliness are maintained across two defensive tiers:

- Client-Side Validation: Uses strict regular expressions to verify student names (/^[a-zA-Z .'-]{2,80}$/), 10-digit numeric phone numbers (/^[0-9]{10}$/), and valid email formats, preventing malformed submissions before HTTP transmission.
- Server-Side Sanitization & Integrity: Re-validates all inputs using PHP filter_var() and preg_match(). It iterates through data.json to enforce email uniqueness, rejecting duplicate student account registrations.

3. Session-Based Authentication & Role Isolation (login.php & admin-login.php):

- Session Fixation Protection: Upon successful credential verification using password_verify() against BCRYPT password hashes, the server executes session_regenerate_id(true) to issue a fresh session token.
- Strict Role Separation: Regular student routes enforce $_SESSION['student_email'] access checks, while privileged administrative controllers (admin-dashboard.php, admin-students.php, admin-events.php) verify $_SESSION['is_admin'] === true, immediately redirecting unauthorized requests.

4. Relational Integrity & Concurrency Control (join.php & LOCK_EX):

- Referential Constraint Checking: When a student requests to join a club, join.php performs a case-insensitive check against the master catalogue (data/clubs.json) to ensure valid foreign-key reference.
- Duplicate Prevention: The server scans memberships.json to prevent duplicate tuples of (email, club_name), redirecting with status=already_joined if already enrolled.
- Concurrency Safety: All file mutation operations employ file_put_contents(..., LOCK_EX), acquiring an exclusive OS-level write lock to protect against race conditions and JSON corruption during concurrent student traffic.

5. Safe DOM Construction & Cross-Site Scripting (XSS) Mitigation:
All dynamic HTML elements are generated using native DOM methods (document.createElement('article'), append()). Text content is assigned via textContent rather than innerHTML, guaranteeing that special characters are automatically sanitized. On the backend, all dynamic values embedded in PHP templates are escaped using htmlspecialchars() to neutralize script injection.

4. CONCLUSION

The Campus ClubJoin Portal successfully addresses the challenge of managing extracurricular student organizations, club discovery, and membership tracking on a modern college campus. By combining clean semantic HTML5 markup, responsive CSS3 stylesheets, reactive client-side JavaScript, and a robust PHP backend with persistent JSON storage, the system delivers a fast, user-friendly, and accessible tool for both students and collegiate administrators.

The application eliminates the overhead, fragmentation, and delays associated with paper sign-up sheets and manual notice boards. Students gain an intuitive self-service portal to explore communities that match their passions, enroll in clubs with a single click, and monitor their active memberships in real time. Simultaneously, campus coordinators obtain an organized administrative dashboard to manage student rosters, schedule campus events, and analyze participation metrics. Overall, the project fulfills its intended mission of improving campus student support services and community engagement through practical, well-architected web technology.

---

5. LEARNING OUTCOMES

Through the design, implementation, and testing of this project, the following core technical and architectural competencies were acquired:

1. Semantic HTML5 & Form Accessibility: Developed accessible, structured web pages utilizing semantic elements (<header>, <nav>, <main>, <article>, <section>, <table>, <form>), custom ARIA attributes, and accessible validation states.

2. Modular CSS3 & UI/UX Principles: Engineered a cohesive, collegiate visual design using CSS variables, Flexbox layouts, responsive CSS Grid, hamburger menu toggles, and accessible contrast ratios without reliance on third-party frameworks.

3. Vanilla JavaScript & Dynamic DOM Programming: Implemented asynchronous HTTP client communication via the Fetch API, real-time multi-criteria filtering, event delegation, and secure DOM manipulation eliminating XSS vectors.

4. Relational File-Based Data Architecture: Designed normalized data schemas (Students, Clubs, Memberships, Events) using structured JSON files, implementing foreign key integrity checks and OS-level file locking (LOCK_EX) for concurrency control.

5. Backend Engineering with PHP & Security Standards: Built modular backend controllers handling stateful session management, cryptographic password hashing (BCRYPT), role-based authorization (RBAC), and sanitization against injection attacks.

6. GITHUB REPOSITORY LINK & DEPLOYMENT

- **GitHub Repository:** https://github.com/Deeppatel016/Club_Join
- **Local Deployment URL:** http://localhost/clubjoin/index.html
- **Student Portal Login URL:** http://localhost/clubjoin/login.html
- **Administrator Portal URL:** http://localhost/clubjoin/admin-login.html (Default Credentials: admin / ClubJoin@123)
- **Web Server Environment:** XAMPP 8.2 (Apache 2.4, PHP 8.2 on Windows 10/11)
- **Data Storage Architecture:** Relational Structured JSON Engine (data.json, data/clubs.json, memberships.json, events.json) with Atomic Concurrency Protection (LOCK_EX)
- **Primary Modules Verified:** index.html, clubs.html, register.html, login.html, dashboard.php, myclubs.php, profile.php, admin-dashboard.php, admin-students.php, admin-events.php

Deployment & Execution Note:
To run the ClubJoin application locally, place the project folder into your web root directory (e.g., C:\xampp\htdocs\ClubJoin). Launch the Apache HTTP Server module from the XAMPP Control Panel and navigate to http://localhost/ClubJoin/ in any standard web browser. Since user credentials, memberships, and events persist dynamically in structured JSON files with atomic OS-level locks, the portal is fully functional out of the box without requiring manual database migration scripts.